CO4-AT3 – Question 28

Name: R.Gautham Krishna

Register No: 192421128

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

Comparative Analysis of Floyd's Algorithm and Bellman-Ford Algorithm

1. Title

Comparative Analysis of Floyd's Algorithm and Bellman-Ford Algorithm for Shortest Path Problems

2. Problem Statement

Given a weighted graph, implement and compare two important shortest-path algorithms:

1. Floyd's Algorithm – All-Pairs Shortest Path
2. Bellman-Ford Algorithm – Single-Source Shortest Path

The objective is to compare their outputs, algorithmic approaches, computational complexity, memory requirements, scalability, and real-world applications.

Floyd's Algorithm calculates the shortest paths between every pair of vertices, while Bellman-Ford calculates the shortest paths from one source vertex to all other vertices.

3. Objective

The objectives of this program are:

- To find all-pairs shortest paths using Floyd's Algorithm.
- To find single-source shortest paths using Bellman-Ford.
- To compare the outputs of both algorithms.
- To understand their algorithmic strategies.
- To analyze their time and space complexity.
- To study their scalability for large graphs.
- To understand their memory requirements.
- To identify suitable real-world applications.
- To provide insights for selecting the appropriate algorithm.

4. Python Program

main.py Visualize Run

```
def floyd_warshall(vertices, edges):
    for u, v, w in edges:
        dist[j][i] = w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )
    return dist
def bellman_ford(vertices, edges, source):
    dist = {v: INF for v in vertices}
    dist[source] = 0
    for _ in range(len(vertices) - 1):
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
            if dist[v] != INF and dist[v] + w < dist[u]:
                dist[u] = dist[v] + w
    return dist
floyd_result = floyd_warshall(vertices, edges)
bellman_result = bellman_ford(vertices, edges, 'A')
print("Floyd's Algorithm - All Pairs Shortest Paths")
for i in range(len(vertices)):
    print(vertices[i], floyd_result[i])
print("\nBellman-Ford - From Source A")
for v in vertices:
    print("A ->", v, "=", bellman_result[v])
```

5.

Sample Output

Output

```
Floyd's Algorithm - All Pairs Shortest Paths
A [0, 3, 2, 8, 10]
B [3, 0, 1, 5, 7]
C [2, 1, 0, 6, 8]
D [8, 5, 6, 0, 2]
E [10, 7, 8, 2, 0]

Bellman-Ford - From Source A
A -> A = 0
A -> B = 3
A -> C = 2
A -> D = 8
A -> E = 10
```

6. Floyd’s Algorithm Analysis

Floyd’s Algorithm, also called the Floyd-Warshall Algorithm, solves the All-Pairs Shortest Path problem.

It calculates the shortest distance between every pair of vertices.

Algorithm Strategy

1. Initialize the distance matrix.
2. Set distance from each vertex to itself as 0.
3. Store the direct edge weights.
4. Consider every vertex as an intermediate vertex.
5. Update the shortest distance.
6. Continue until all vertices are considered.

The main update is:

dist[i][j] =

min(dist[i][j], dist[i][k] + dist[k][j])

7. Bellman-Ford Algorithm Analysis

Bellman-Ford solves the Single-Source Shortest Path problem.

It calculates the shortest distance from one source vertex to all other vertices.

In this example, the source is:

A

Algorithm Strategy

1. Initialize the source distance to 0.
2. Set all other distances to infinity.
3. Relax all edges repeatedly.
4. Repeat the relaxation V − 1 times.
5. Check for negative-weight cycles.

The relaxation operation is:

```
if dist[u] + weight < dist[v]:
    dist[v] = dist[u] + weight
```

8. Comparison Between Floyd and Bellman-Ford

Feature	Floyd’s Algorithm	Bellman-Ford
Problem Type	All-Pairs Shortest Path	Single-Source Shortest Path
Output	Distance between every pair	Distance from one source
Approach	Dynamic Programming	Repeated Edge Relaxation
Negative Edges	Supported	Supported
Negative Cycle Detection	Yes	Yes
Time Complexity	$O(V^3)$	O(VE)
Space Complexity	$O(V^2)$	O(V)
Best For	Dense/small graphs	Single-source problems
Memory Usage	High	Low
Scalability	Limited for very large graphs	Better for sparse graphs
Source Vertex	Not required	Required

9. Shortest Path Output Comparison

For source vertex A, both algorithms produce:

Destination	Floyd’s Algorithm	Bellman-Ford
A	0	0
B	3	3
C	2	2
D	8	8
E	10	10

Therefore, both algorithms correctly identify the shortest distances from A.

However, Floyd’s Algorithm also calculates distances such as:

B → E = 7

C → E = 8

D → A = 8

Bellman-Ford does not calculate all of these in a single execution because it starts from one source.

10. Computational Complexity

Floyd's Algorithm

Floyd's Algorithm uses three nested loops:

for k

  for i

    for j

Therefore:

Time Complexity – $O(V^3)$

Space Complexity – $O(V^2)$

Where:

- V = Number of vertices

Bellman-Ford Algorithm

Bellman-Ford relaxes all edges $V - 1$ times.

Therefore:

Time Complexity – O(VE)

Space Complexity – O(V)

Where:

- V = Number of vertices
- E = Number of edges

11. Scalability for Large Graphs

Floyd's Algorithm

Floyd's Algorithm becomes expensive when the number of vertices increases because its time complexity is:

$O(V^3)$

For a graph with thousands of vertices, the number of calculations becomes extremely large.

Therefore, it is generally more suitable for small or moderately sized graphs, especially when shortest paths between many pairs are required.

Bellman-Ford

Bellman-Ford has:

O(VE)

complexity.

For a sparse graph where E is relatively small, it can be more practical than Floyd's Algorithm.

However, for extremely large graphs, Bellman-Ford can also become computationally expensive.

12. Memory Requirements

Floyd's Algorithm

Floyd maintains a complete distance matrix:

$V \times V$

Therefore:

Space Complexity – $O(V^2)$

For a large graph, this requires significant memory.

Bellman-Ford

Bellman-Ford mainly stores the distance of each vertex:

dist[V]

Therefore:

Space Complexity – O(V)

This makes Bellman-Ford more memory-efficient.

13. Negative Edge Handling

One important advantage of both algorithms is their ability to handle negative edge weights.

For example:

$A \rightarrow B = 4$

$B \rightarrow C = -2$

Bellman-Ford can correctly calculate shortest paths even when negative edges are present.

Floyd's Algorithm can also handle negative edges.

However, both algorithms need to account for negative-weight cycles, because a shortest path is not well-defined if a reachable negative cycle can be repeatedly traversed.

14. Algorithm Selection

The selection of the algorithm depends on the application.

Use Floyd's Algorithm when:

- Shortest paths between all pairs are required.
- The graph is relatively small.
- The graph is dense.
- Multiple shortest-path queries are expected.

Use Bellman-Ford when:

- Only one source vertex is important.
- The graph contains negative edge weights.
- Negative cycle detection is required.
- Memory usage needs to be lower.

15. Real-World Applications

Floyd's Algorithm

Floyd's Algorithm can be used for:

- Network routing analysis
- Transportation planning
- City-to-city distance analysis
- Communication network optimization
- Finding shortest paths between multiple locations

Example

A logistics company wants to know the shortest transportation distance between every warehouse and every distribution center.

Floyd's Algorithm can calculate all these distances simultaneously.

Bellman-Ford Algorithm

Bellman-Ford can be used for:

- Network routing
- Financial transaction analysis

- Transportation networks
- Routing protocols
- Graphs containing negative costs

Example

A network system needs to calculate the shortest route from one server to all other servers.

Bellman-Ford can calculate the shortest paths starting from that server.

16. Graph Optimization Insights

Both algorithms solve shortest-path problems, but their objectives differ.

Floyd's Algorithm

Optimizes:

Shortest paths between every pair of vertices.

Bellman-Ford

Optimizes:

Shortest paths from one selected source vertex.

Therefore:

Floyd = All-Pairs

Bellman-Ford = Single-Source

Floyd requires more memory because it stores the complete distance matrix.

Bellman-Ford requires less memory but must be executed again if a different source vertex is selected.

17. Advantages

Floyd's Algorithm

- Computes all-pairs shortest paths.
- Simple implementation.
- Supports negative edge weights.
- Useful when many shortest-path queries are required.
- Easy to represent using a distance matrix.

Bellman-Ford

- Computes single-source shortest paths.

- Supports negative edge weights.
- Can detect negative-weight cycles.
- Requires less memory.
- Useful for sparse graphs and single-source applications.

18. Implementation Challenges

1. Large Number of Vertices

Floyd's $O(V^3)$ complexity can become expensive.

Solution:

Use other shortest-path algorithms when appropriate.

2. Negative Cycles

Negative cycles can make shortest paths undefined.

Solution:

Perform negative-cycle detection before interpreting the result.

3. Large Graph Memory

Floyd requires an $O(V^2)$ distance matrix.

Solution:

Use Bellman-Ford or other graph representations when only single-source results are required.

4. Changing Source

Bellman-Ford must be executed again for another source.

Solution:

Use Floyd when shortest paths from many different sources are required.

19. Result Interpretation

The program successfully calculated shortest paths using both algorithms.

For source vertex A:

A → A = 0

A → B = 3

A → C = 2

A → D = 8

A → E = 10

Both algorithms produce the same shortest distances from A.

The major difference is that Floyd's Algorithm calculates the shortest distance between every pair of vertices, while Bellman-Ford calculates shortest distances from one source vertex.

20. Algorithm Selection Insights

The following decision can be used:

Need shortest paths between all pairs?

|

YES

↓

Floyd's Algorithm

NO

↓

Need shortest paths from one source?

|

YES

↓

Bellman-Ford

For small graphs requiring many pair-to-pair queries, Floyd is a good choice.

For single-source shortest paths with negative edges, Bellman-Ford is more appropriate.

For very large graphs, neither should automatically be assumed best; graph density, edge weights, number of queries, and whether negative edges exist should guide the choice.

21. Conclusion

Floyd's Algorithm and Bellman-Ford are important shortest-path algorithms with different purposes.

Floyd’s Algorithm solves the All-Pairs Shortest Path problem and calculates the shortest distance between every pair of vertices. Its time complexity is $O(V^3)$ and space complexity is $O(V^2)$.

Bellman-Ford solves the Single-Source Shortest Path problem and calculates the shortest distances from one source vertex. Its time complexity is $O(VE)$ and space complexity is $O(V)$.

Therefore, Floyd’s Algorithm is suitable when all-pairs shortest paths are required, whereas Bellman-Ford is suitable when shortest paths from a single source are required, especially when negative edge weights are present.

Final Complexity

Floyd’s Algorithm: Time Complexity – $O(V^3)$ | Space Complexity – $O(V^2)$

Bellman-Ford: Time Complexity – $O(VE)$ | Space Complexity – $O(V)$